Machine Learning-Based Bestseller Prediction on Amazon Product Dataset

Subject: Big Data & AI

Submitted to: Madan Radhahkrishnan

By: Muhammad Jamal Anjum

(jamal.anjum2001@gmail.com)

Session: ADEO2 (2025-2026)

Master 2 in Data Mining and Optimization architecture

Amazon Product Churn

(Finding the bestseller Product)

1. Describe the problem statement and objective

Problem Statement

In modern e-commerce platforms, identifying potential best-selling products at an early stage is a critical challenge for businesses. Best-selling products contribute significantly to revenue generation, inventory planning, marketing strategies, and customer satisfaction. However, due to the large volume of products and rapidly changing customer preferences, manual identification of best sellers is inefficient and error-prone.

This project aims to address this challenge by leveraging Big Data and Machine Learning techniques to predict whether a product will become a best seller based on its attributes such as price, ratings, number of reviews, category, and recent purchase behavior. Using a scalable PySpark-based machine learning pipeline, the project focuses on building a classification model capable of accurately predicting the bestseller status of products in an e-commerce marketplace.

Project Objectives

The primary objective of this project is to develop, optimize, and evaluate a robust **Binary Classification Model** using an e-commerce product dataset to predict the **Best Seller status** of products. The model aims to accurately distinguish between products that are likely to become best sellers and those that are not, based on product characteristics, pricing information, customer feedback, and recent purchase behaviour.

- **Data Preparation:**
  Construct a unified analytical dataset by integrating product and category data, followed by comprehensive data pre-processing and feature engineering. This includes handling missing values, encoding categorical variables, creating derived features such as price discounts, and addressing potential class imbalance in the target variable.

- **Model Development:**
  Implement, compare, and analyze the performance of a minimum of five distinct classification algorithms in accordance with project requirements. These models will first be trained without optimization, followed by training the top-performing models using Cross-Validation and Grid Search techniques to improve predictive performance.

- **Performance Benchmarking:**
  Evaluate all trained models using standard classification metrics on a holdout test dataset. The final selected model should demonstrate strong predictive capability and generalization performance, providing sufficient confidence for real-world application.

- **Deployment Design:**
  Propose three to five scalable system architecture designs leveraging modern cloud and big data platforms such as AWS, GCP, Azure, or Databricks to deploy the final PySpark-based model for efficient batch and/or real-time inference.

2. Ideal Production Architectures for Deployment

This section examines the industry-standard lifecycle for building and deploying Machine Learning models at scale. For e-commerce giants like Amazon, traditional single-machine tools (like Pandas) are insufficient. Instead, **Apache Spark (PySpark)** is the primary solution for handling the "Three Vs" of Big Data: Volume, Velocity, and Variety.

1. The Necessity of PySpark for E-commerce Analytics

In this project, we analyze thousands of products across diverse categories.

- **Scalability:** Unlike Scikit-Learn, PySpark distributes the dataset across a cluster, meaning if our product list grows from 10,000 to 10 million, the system remains stable.

- **Unified Engine:** PySpark allows us to use the same engine for Data Ingestion (SQL), Feature Engineering, and Machine Learning (MLlib), reducing the complexity of the "tech stack.

2. The Model Building Lifecycle: A PySpark Approach

- **Distributed Loading:** Data is loaded from Amazon S3 or HDFS into a Spark DataFrame. PySpark's schema inference allows it to automatically detect data types for columns like price (float) and stars (double).

- **Table Joining:** In our project, we perform a **Broadcast Join** between the Product table and the Category table. This is a specific Spark optimization where the smaller table (Categories) is sent to all worker nodes to make the join operation significantly faster.

- **Handling Categorical Data:** We use StringIndexer to convert category_name into numerical indices. For high-cardinality categories, existing solutions might use Feature Hashing.

- **The VectorAssembler:** A unique requirement of PySpark is that all input features (Price, Reviews, Stars) must be merged into a single "Feature Vector" column. This is a critical step that allows Spark to perform distributed linear algebra.

- **Normalization:** Since reviews might be in the thousands and stars are between 1 and 5, we use StandardScaler to ensure that features with larger numbers don't unfairly bias the model

3. Summary of the Solution

By using PySpark, we bridge the gap between "Data Science" and "Data Engineering." The solution is not just a mathematical model, but a **scalable pipeline** capable of processing millions of Amazon product records, transforming them into insights, and delivering those insights into a production database for business use.

3. Ideal Production Architectures for Deployment

A production architecture for Big Data ML must be scalable, fault-tolerant, and capable of handling distributed computing. Below are the designs for the leading cloud platforms.

3.1 Architecture 1: Databricks "Lakehouse" Architecture (Unified Analytics)

Databricks is the industry leader for Spark-based projects. This architecture uses the **Medallion Design** to refine data quality.

- **Data Lake (Bronze Layer):** Raw Amazon CSV files are ingested into Delta Lake.
- **Refinement (Silver Layer):** PySpark is used to join the Product and Category tables
- and clean data (handling missing prices or titles).
- **Feature Store:** Features like "Average Rating per Category" are stored in the Databricks Feature Store for reuse.
- **MLflow Tracking:** All 5 classification models are tracked here to compare AUROC and Accuracy.
- **Model Serving:** The best model is deployed as a **REST API endpoint** directly within Databricks.

3.2 Architecture 2: AWS Cloud-Native (Scalable & Managed)

This architecture is ideal for high-velocity e-commerce data using Amazon’s managed services.

- **Storage:** Data is stored in **Amazon S3** (Simple Storage Service).
- **Cataloging: AWS Glue** crawls the data to create a metadata catalog and runs Spark ETL jobs.
- **Development: Amazon SageMaker Studio** provides the environment for PySpark notebook developmentz
- **Distributed Training:** SageMaker spins up a transient cluster to train the Random Forest/GBT models using Spark MLlib and saves the model artifacts back to S3.
- **Deployment:** The model is hosted on **SageMaker Endpoints** with Auto-Scaling enabled to handle fluctuating traffic.

4. Tools & Technologies Used

This project was implemented using Big Data and machine learning tools designed to handle large-scale datasets efficiently. Apache Spark was selected due to its distributed computing capabilities and in-memory processing, which significantly improves performance when working with large datasets.

Technologies used:

- **Apache Spark** (PySpark)
- **Python**
- **Pandas & Matplotlib** (for lightweight visualization)
- **Visual studio code** (VS Code)

5. Why PySpark?

PySpark was chosen for this project because it enables scalable data processing and machine learning on large datasets. It supports distributed data transformations, built-in machine learning pipelines, and seamless integration with Python, making it suitable for end-to-end Big Data analytics.

6. Libraries Used:

The project uses PySpark for large-scale data processing and machine learning, along with Python libraries for analysis and visualization. The main libraries used are listed below along with their purpose.

PySpark Machine Learning Libraries:

- **Pipeline**: Used to structure data preprocessing and model training into a single workflow.
- **VectorAssembler**: Combines multiple feature columns into a single feature vector required for machine learning models.
- **StandardScaler**: Scales numerical features to ensure consistent contribution to the model.
- **LogisticRegression**: Applied as a classification algorithm to predict whether a product is a Best Seller.
- **BinaryClassificationEvaluator**: Evaluates model performance using metrics such as Area Under the ROC Curve (AUC).
- **MulticlassClassificationEvaluator**: Used to compute additional classification metrics such as accuracy, precision, and recall.

Data Analysis and Visualization Libraries:

- **Pandas** and **NumPy**: Used for data manipulation and numerical operations after converting Spark DataFrames when required.
- **Matplotlib** and **Seaborn**: Used for visualizing model results and key data insights.

7. Dataset Description

This project uses a large-scale Amazon e-commerce dataset consisting of approximately 1.4 million products. The dataset contains detailed information about product listings, customer feedback, pricing, and sales activity collected in September 2023.

The data is provided in two separate files: amazon_products.csv and amazon_categories.csv, which are combined to create a unified analytical dataset.

Product Dataset (amazon_products.csv):

The product dataset contains detailed information related to individual Amazon products. Each record represents a single product along with its attributes and sales indicators.

- **asin**: Unique product identifier
- **title**: Product title
- **imgUrl**: URL of the product image
- **productURL**: URL of the product page
- **stars**: Average customer rating
- **reviews**: Number of customer reviews
- **price**: Current product price
- **listPrice**: Original listed price
- **category_id**: Category identifier
- **isBestSeller**: Target variable indicating whether a product is a Best Seller
- **boughtInLastMonth**: Number of times the product was purchased in the last month

Category Dataset (amazon_categories.csv)

The category dataset provides descriptive information about product categories available on Amazon.

- **id**: Unique category identifier
- **category_name**: Name of the product category

Data Integration (Joining Datasets)

The **amazon_products** and **amazon_categories** datasets are linked through a foreign key relationship. The category_id column in the product dataset references the id column in the category dataset.

Using this relationship, both datasets were joined to associate each product with its corresponding category name. This integration enriches the product data and enables category-level analysis and modeling.

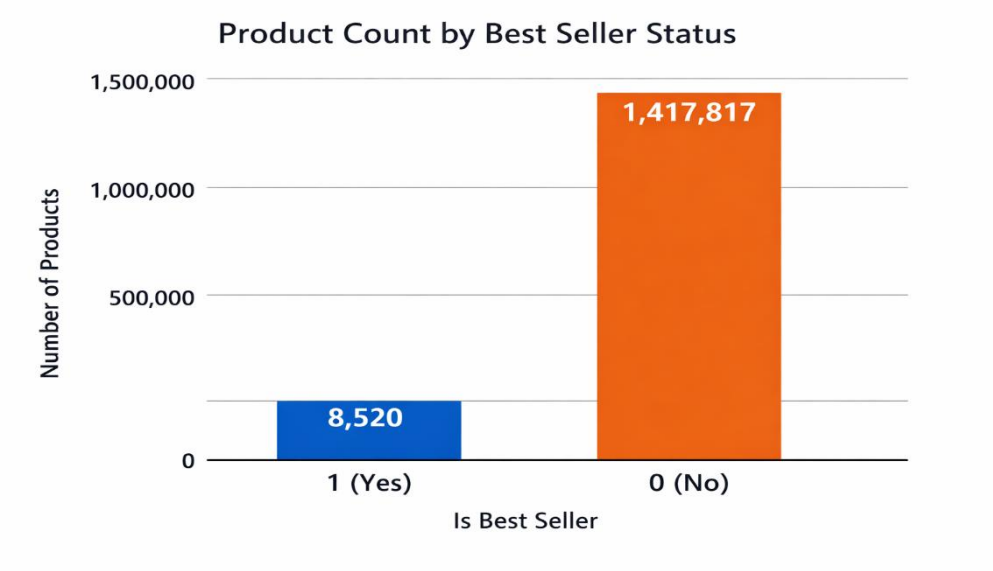
8. Target Variable

The target variable for this project is **isBestSeller**, a binary attribute that indicates whether a product is classified as a Best Seller. This variable is used to build and evaluate classification models.

9. EDA & Pre-processing

During the exploratory data analysis (EDA) phase, the distribution of the target variable **isBestSeller** was examined. The analysis revealed a **significant class imbalance** in the dataset.

The number of non–Best Seller products was much higher compared to Best Seller products. This imbalance indicates that the dataset is dominated by one class, which may negatively affect the performance of machine learning models.



Impact of Class Imbalance:

A highly imbalanced dataset can lead to model bias, where the classifier tends to favor the majority class. As a result, the model may achieve high accuracy while failing to correctly predict the minority class.

Additionally, class imbalance increases the risk of **overfitting**, as the model may learn patterns that do not generalize well to unseen data.

<u>Handling the imbalance Dataset:</u>

The dataset shows a significant class imbalance, with a very high number of No (isBestSeller = 0) products compared to Yes (isBestSeller = 1). To prevent overfitting and bias in the model, it is necessary to balance the dataset.

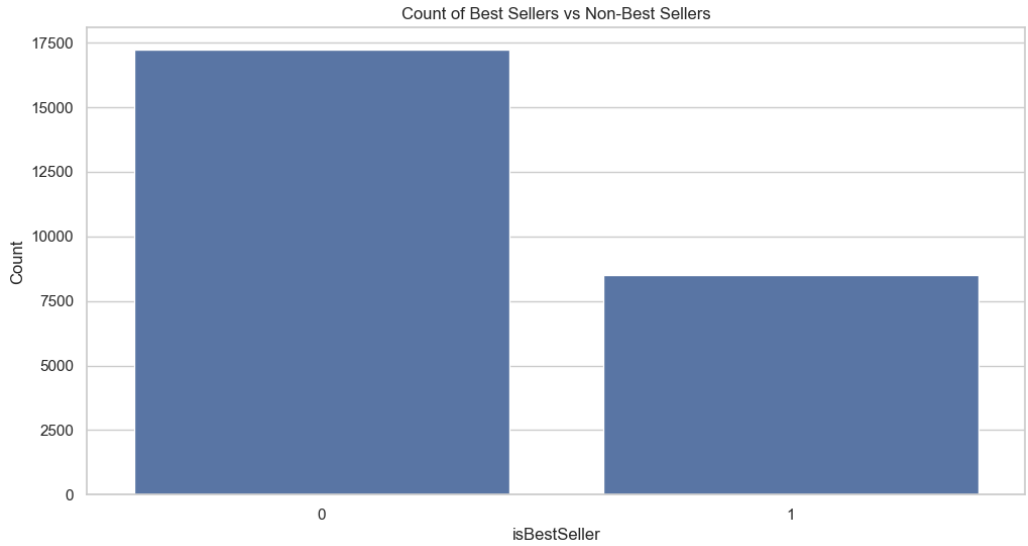
Due to machine limitations and large dataset size, we opted for **downsampling** the majority class (No). Specifically, we randomly sampled the No class to be **twice the size of the Yes class**, ensuring a more manageable dataset size while maintaining sufficient information for model training.

Other approaches to handle imbalance include:

- **Upsampling the minority class (Yes)**
- **Synthetic data generation (SMOTE, ADASYN)**

- **Using class weights in model training**

For this project, downsampling was chosen as the most practical solution given the dataset size and system constraints.

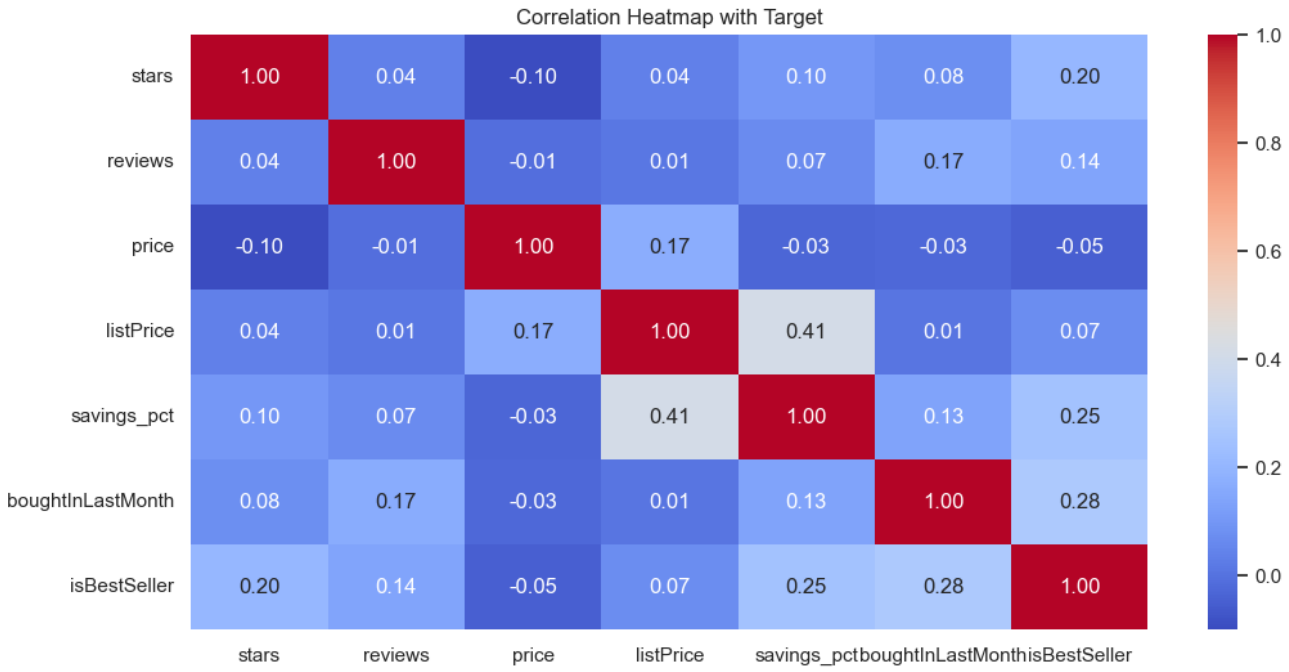


Correlation with Target:

To understand how each numerical feature relates to the target variable (isBestSeller), we calculated the **correlation** between the numerical columns and the target.

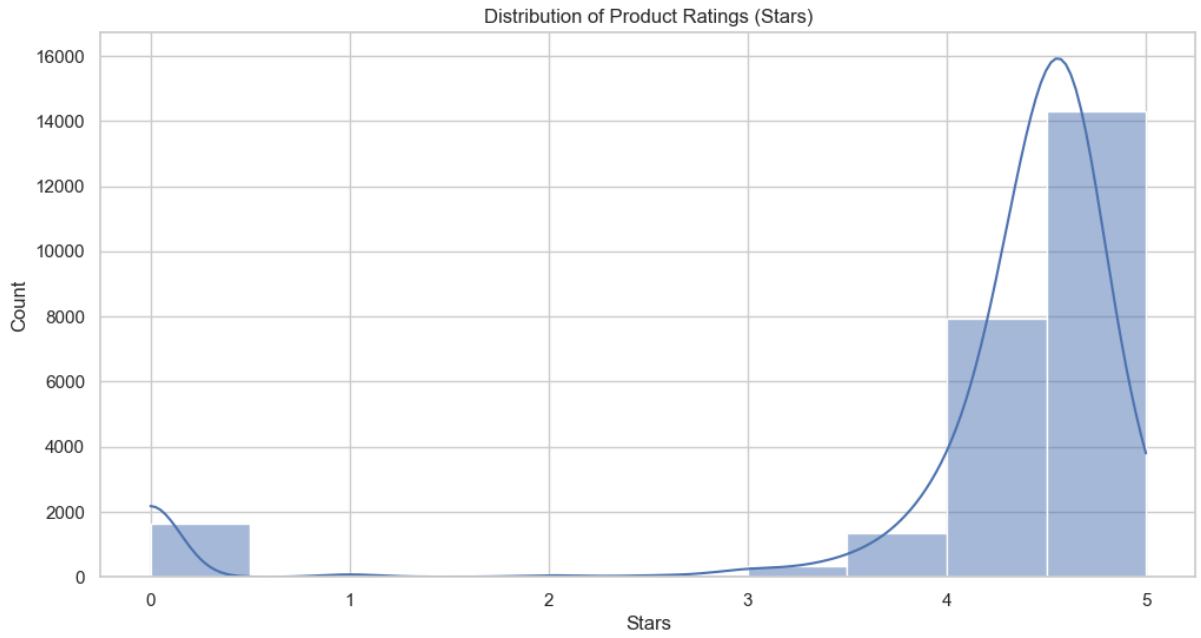
Correlation analysis helps to:

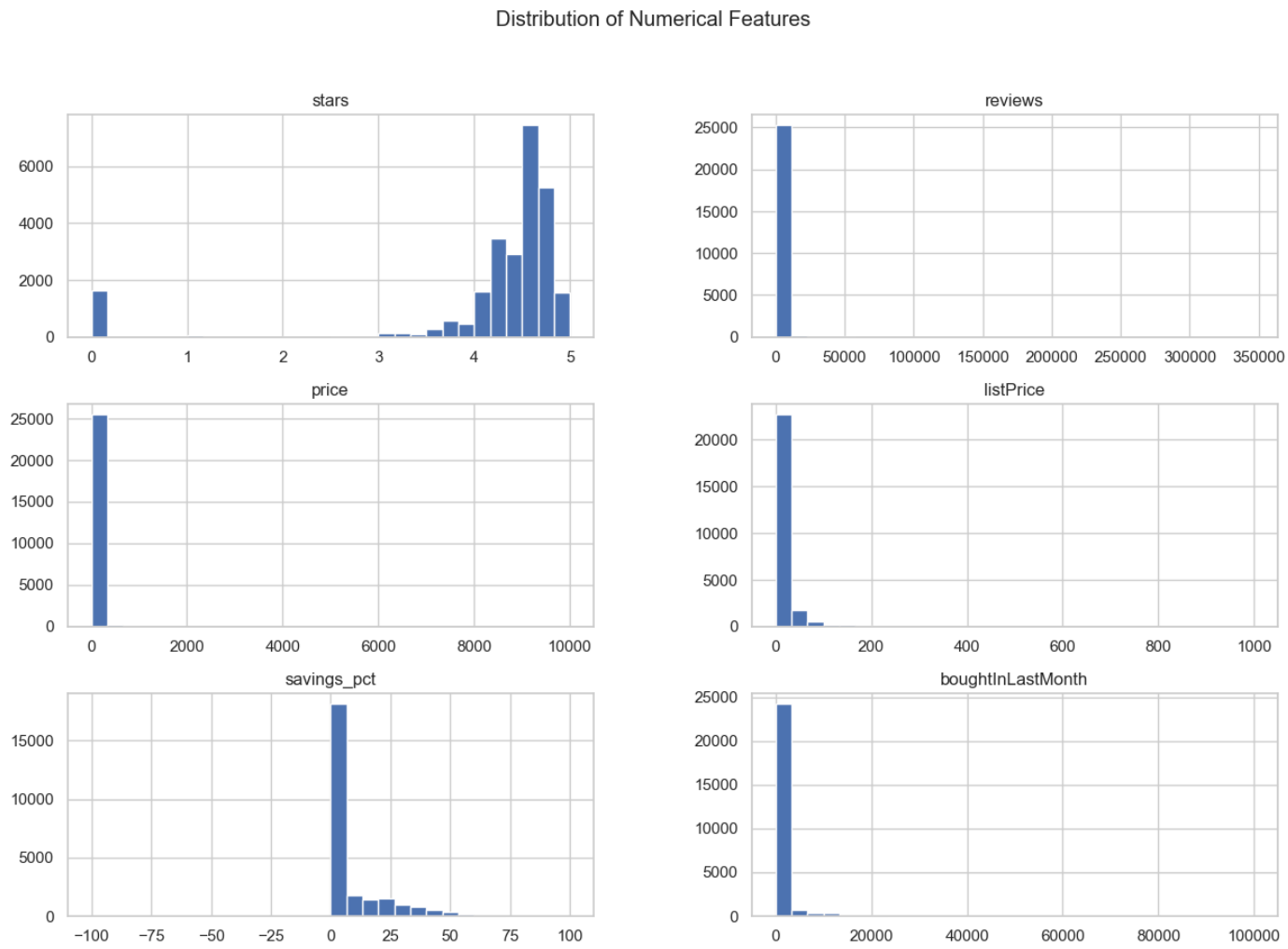
- Identify features that have a strong relationship with the target.
- Determine which variables are likely to be more predictive.
- Detect potential redundancy among features.



Star Distribution:

The figure below is a visual representation of the distribution of product ratings (stars) in the dataset. It shows that the majority of products have high ratings, with a significant concentration around **5 stars**. This indicates that most products in the dataset are highly rated, which could influence the prediction of the isBestSeller target, as higher-rated products are more likely to become bestsellers.





Other Features' Visualization:

The dataset initially contains a limited number of features for predicting isBestSeller. To improve the predictive power of our model, we created **additional derived features** that are relevant to a product's sales performance.

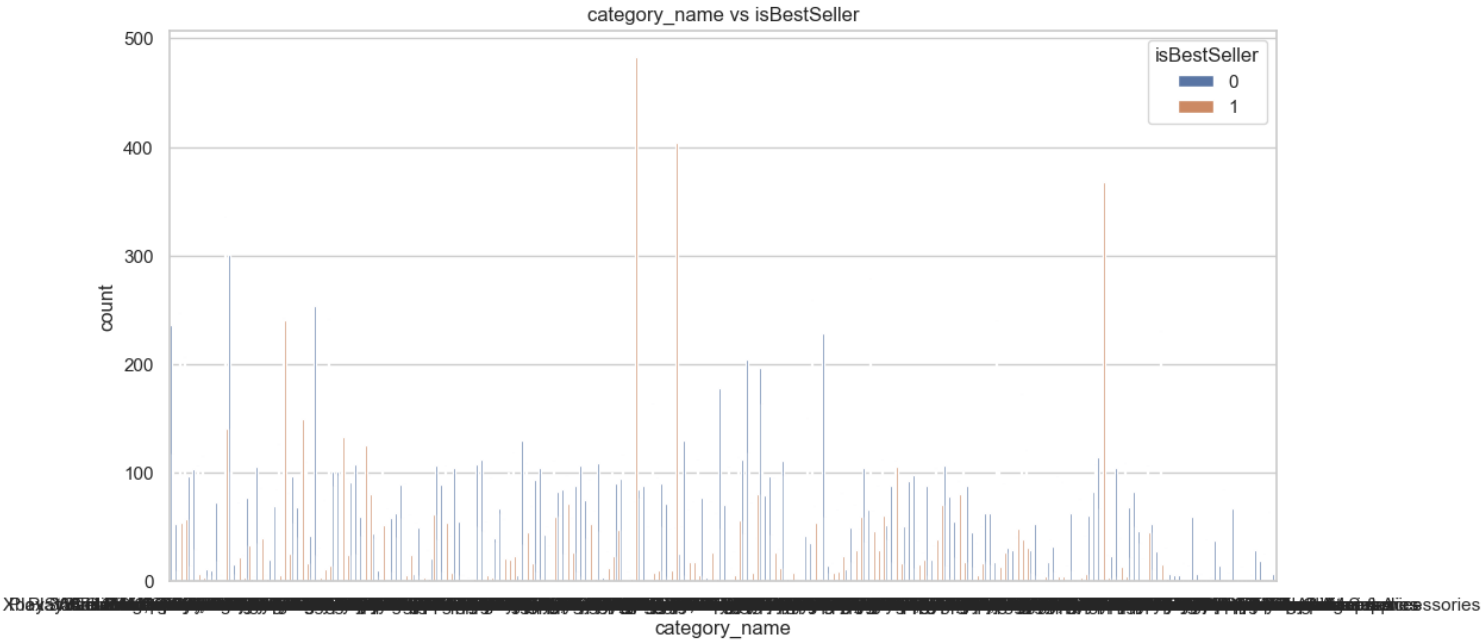
When considering why a product sells, key factors include:

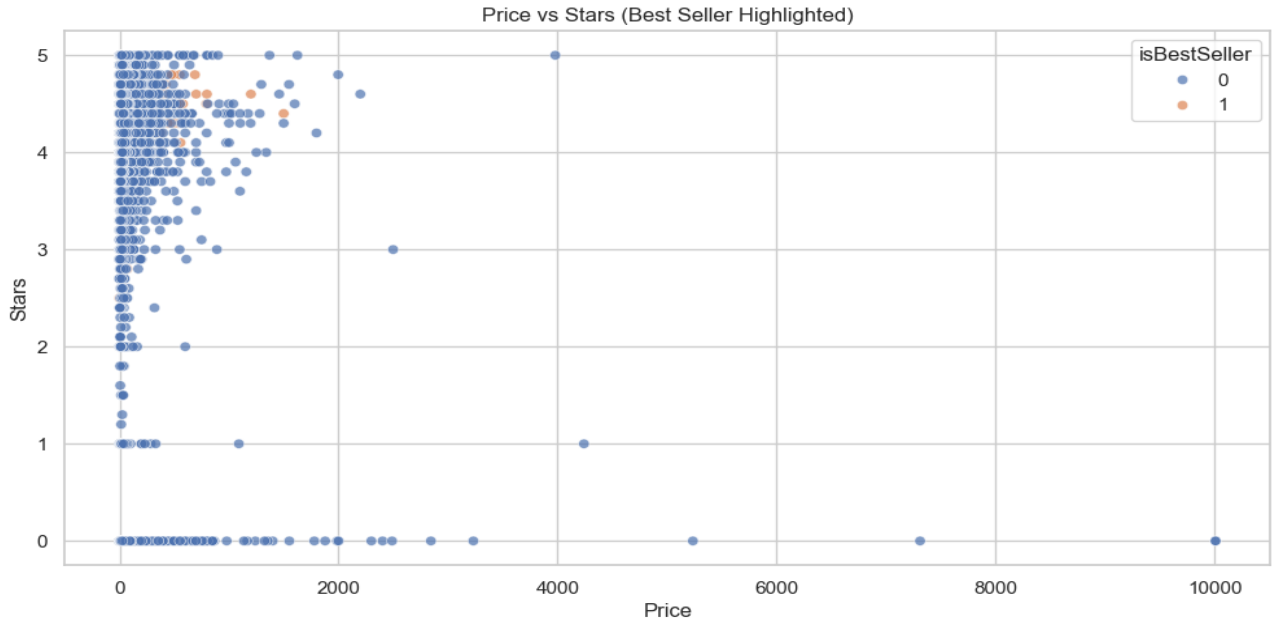
- **Price of the product**
- **Availability of discounts**
- **Customer reviews and ratings**

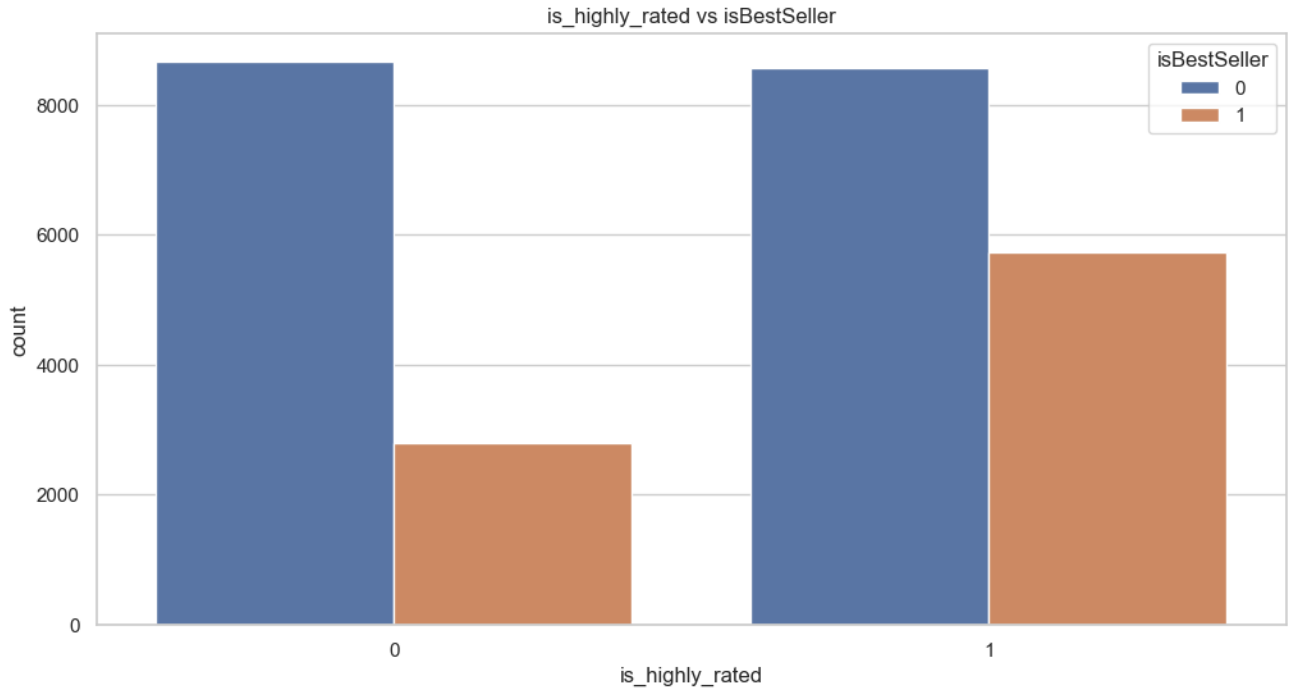
Based on these insights, we derived the following features:

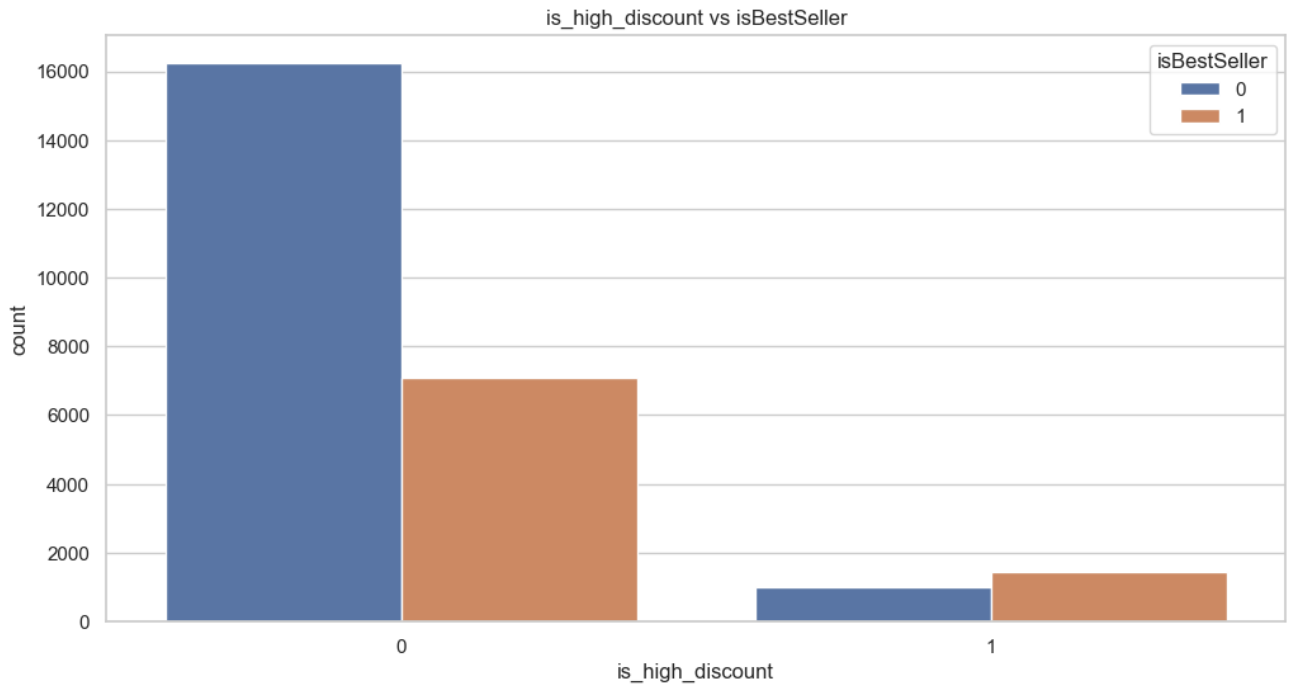
1. **is_high_discount** – Indicates whether the product has a significant discount.
2. **is_highly_rated** – Indicates whether the product has high customer ratings.
3. **savings_pct** – Represents the percentage saved from the original price (listPrice vs price).

These features are extracted from the original price and listPrice columns and are intended to provide additional predictive signals for identifying potential bestsellers.







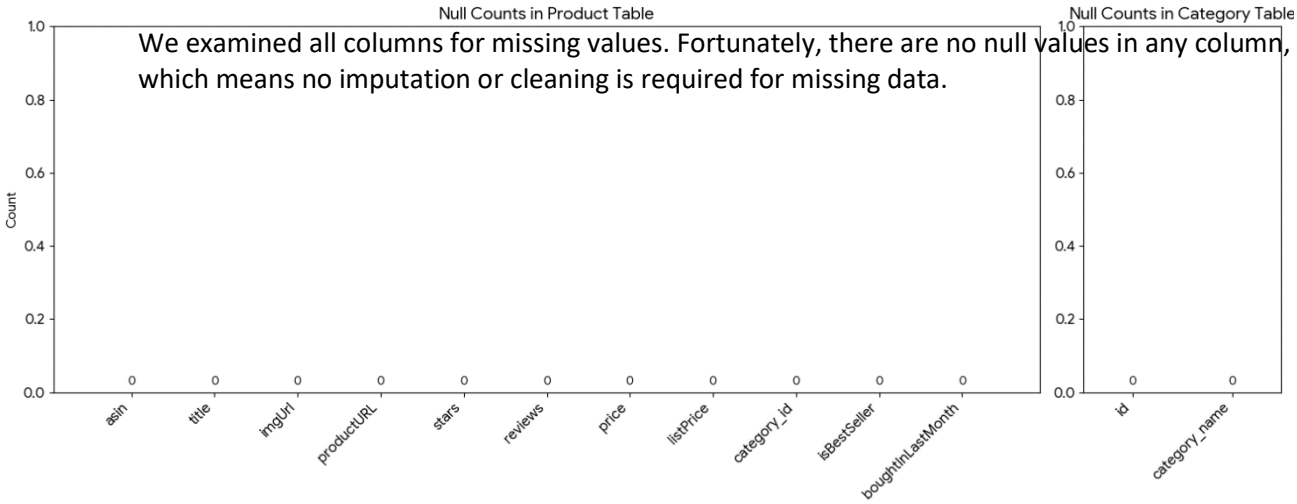


10. Data Pre-Processing:

Data preprocessing is a crucial step to ensure the dataset is clean and ready for machine learning. In our analysis, we first checked for null values and found that all columns were complete, requiring no imputation. Next, we examined the unique values in each column to understand the data distribution and finalize relevant features, selecting isBestSeller as our target variable. We also checked for duplicate rows and confirmed that none were present, eliminating any redundancy. With these steps completed, the dataset is clean, consistent, and ready for model training.

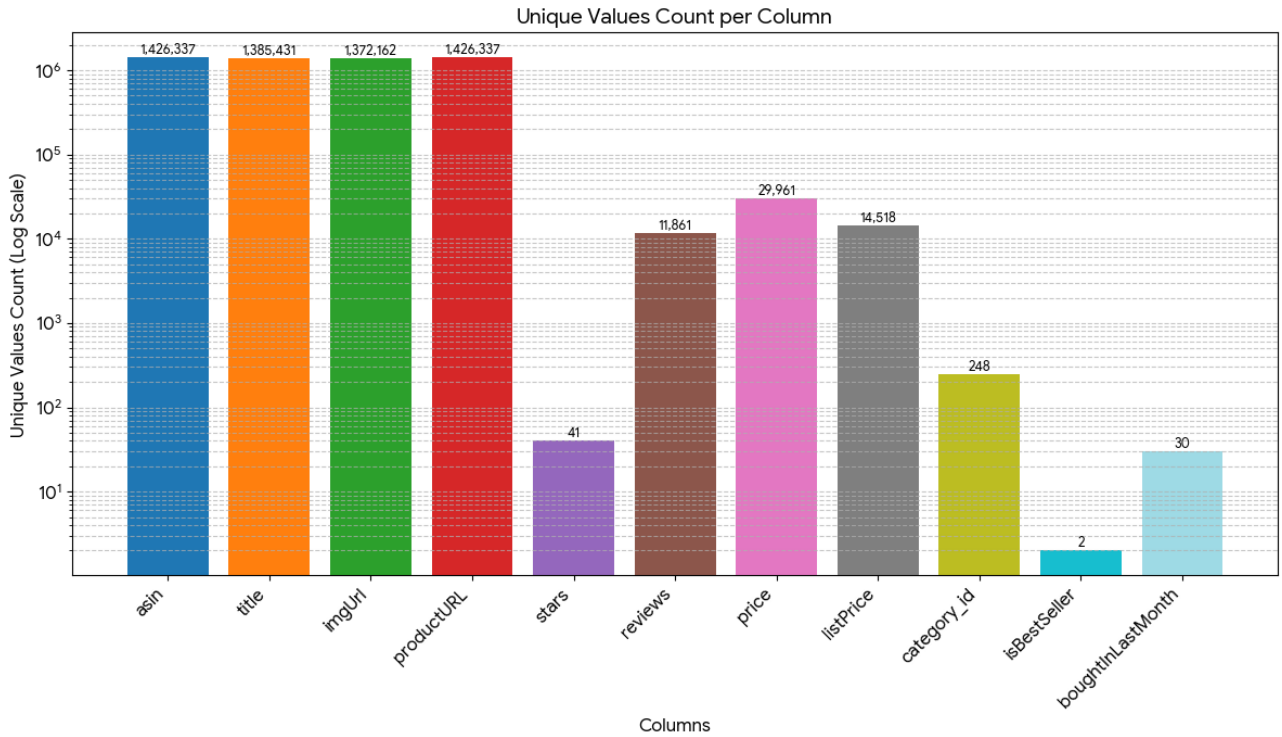
- **Checking for Null Values:**

We examined all columns for missing values. Fortunately, there are no null values in any column, which means no imputation or cleaning is required for missing data.



- **Analyzing Unique Values:**

  We explored the unique values in each column to understand the data distribution and determine which columns are relevant for prediction. Based on this analysis, we finalized **isBestSeller** as our target variable.



- **Checking for Duplicate Records:**

  Duplicate rows can cause redundancy and bias in model training. Upon inspection, there were no duplicate rows in the dataset.

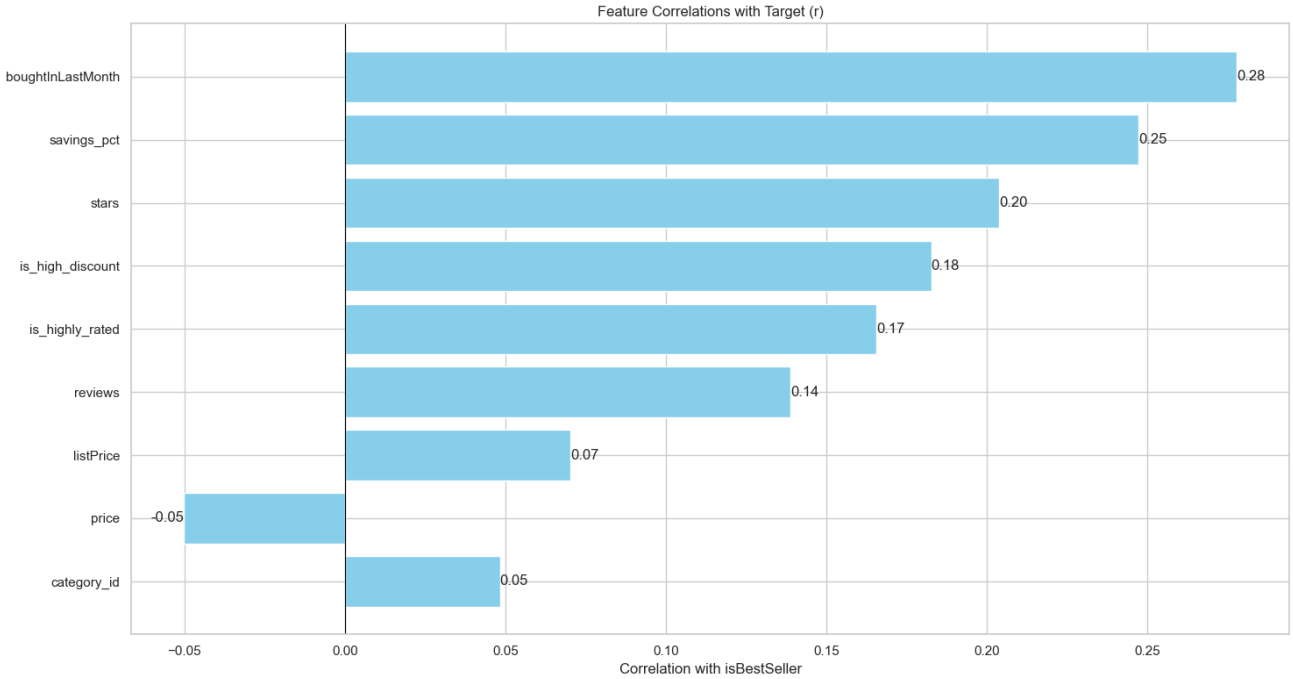
- **Correlated Features for ML:**

  To identify which features have the strongest relationship with the target variable (isBestSeller), we computed the **feature correlations** in the dataset. Correlation analysis helps determine the **predictive importance** of each feature and can guide feature selection for model training.

The figure below illustrates the correlation of different features with the target. Features with higher correlation values are more likely to influence the model's prediction. From the analysis, the following features were found to be significantly correlated with isBestSeller:

- Stars
- Reviews
- listPrice
- boughtInLastMonth
- savings_pct
- is_high_discount
- is_highly_rated

These correlated features provide strong signals for predicting bestsellers and will be used in the machine learning models.



11. Model Training

After preprocessing the dataset and selecting the most relevant features, we proceeded to train multiple machine learning models to predict whether a product is a bestseller (isBestSeller). Since this is a **binary classification problem**, we explored a variety of algorithms to identify the most effective model for our dataset.

The models used for training include:

- **Logistic Regression:** A simple yet effective linear model for binary classification, useful as a baseline.
- **Decision Tree:** Captures non-linear relationships between features and the target, offering interpretability.
- **Random Forest:** An ensemble of decision trees that reduces overfitting and improves generalization.
- **Gradient Boosting:** Sequential ensemble method that focuses on minimizing prediction errors, often yielding high accuracy.

- **Neural Network:** A deep learning approach capable of capturing complex patterns and interactions in the data.

We trained each model using the selected features: stars, reviews, listPrice, boughtInLastMonth, savings_pct, is_high_discount, and is_highly_rated. The performance of these models was evaluated to identify which algorithm best predicts bestseller products, considering both accuracy and the ability to handle imbalanced data.

Data Preparation and Pipeline:

To ensure robust model training and evaluation, we first **split the dataset** into **80% training** and **20% testing** subsets. This allows us to train the models on the majority of the data while keeping a separate portion for evaluating their performance.

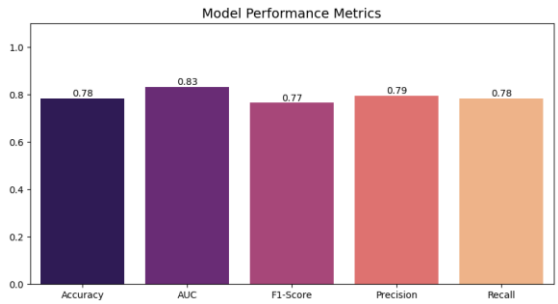
Next, we prepared the features for machine learning models using PySpark:

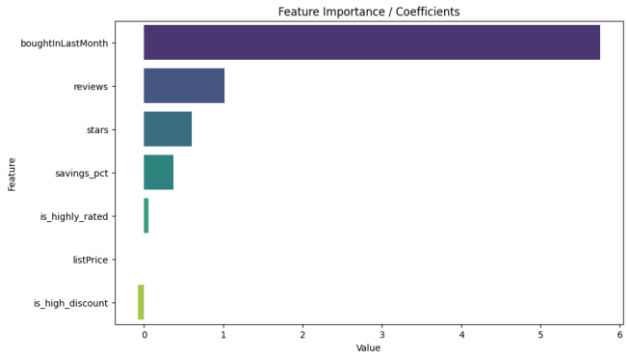
1. **VectorAssembler:** All selected features (stars, reviews, listPrice, boughtInLastMonth, savings_pct, is_high_discount, is_highly_rated) were combined into a single feature vector. This step converts multiple feature columns into a format suitable for ML algorithms in PySpark.
2. **StandardScaler:** The feature vectors were scaled to have zero mean and unit variance. Scaling ensures that features with larger numeric ranges do not dominate the learning process, improving model performance.
3. **Pipeline Creation:** A standard PySpark pipeline was created to encapsulate all preprocessing steps (VectorAssembler + StandardScaler) along with the model training step. Using a pipeline ensures reproducibility and a clean workflow, simplifying model training and evaluation.

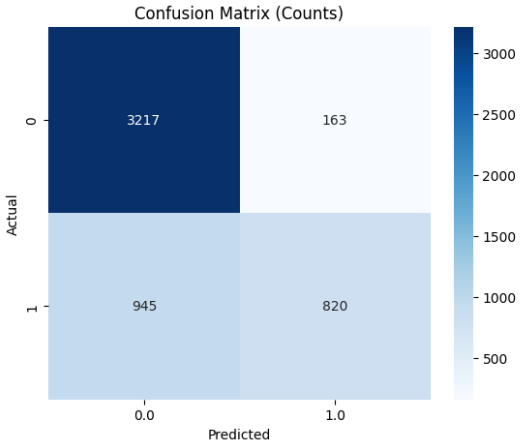
This structured preparation allows the models to learn efficiently from the dataset and ensures consistent preprocessing across both training and testing sets.

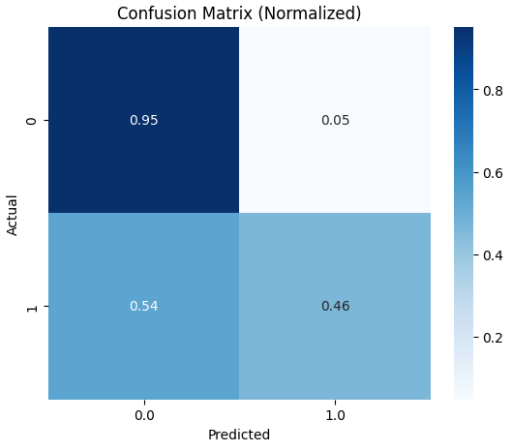
Now below we will see the Performance of different models and there AUC and F1 Score

Logistic Regression:

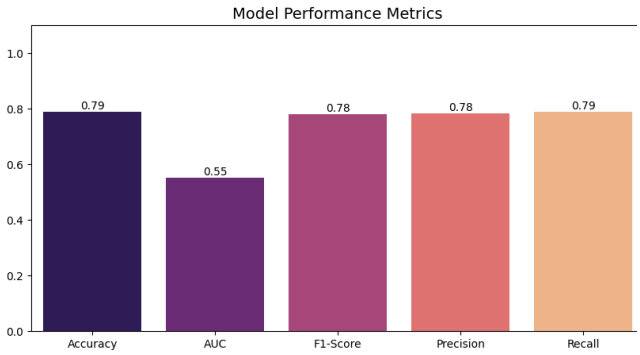


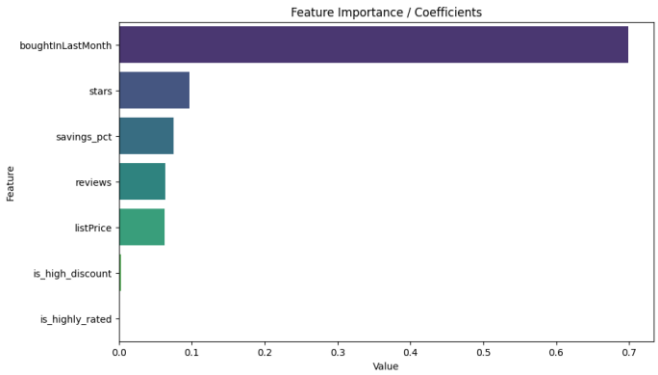


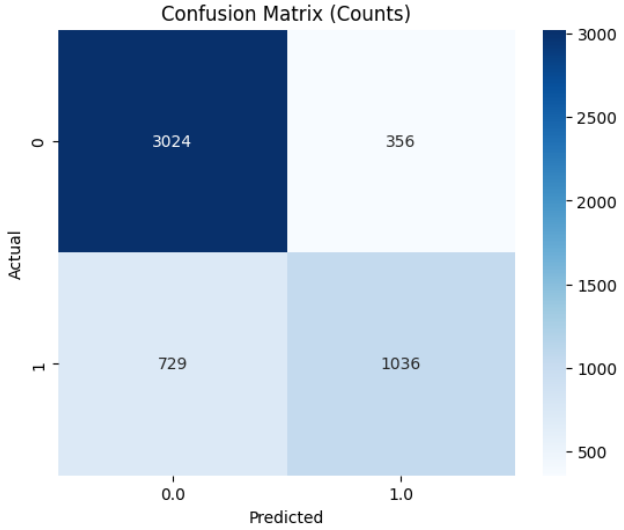


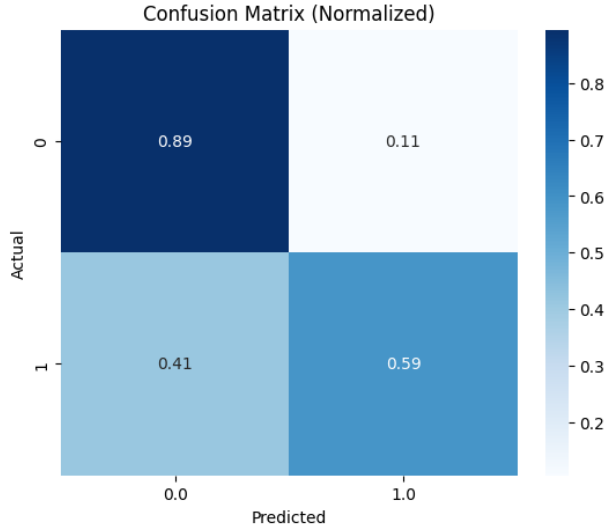


Decision Tree Classifier:

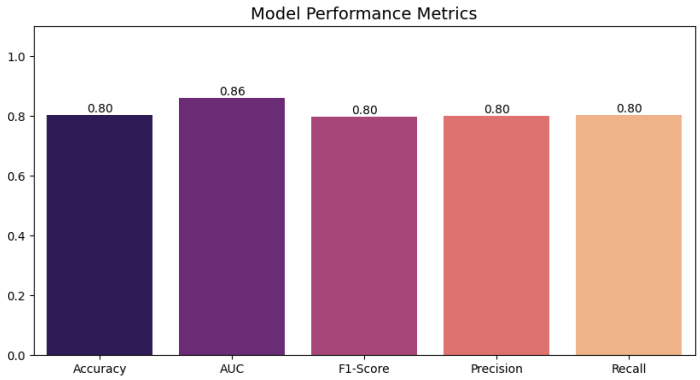


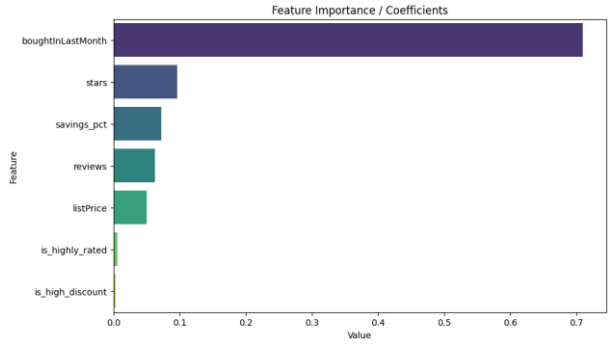


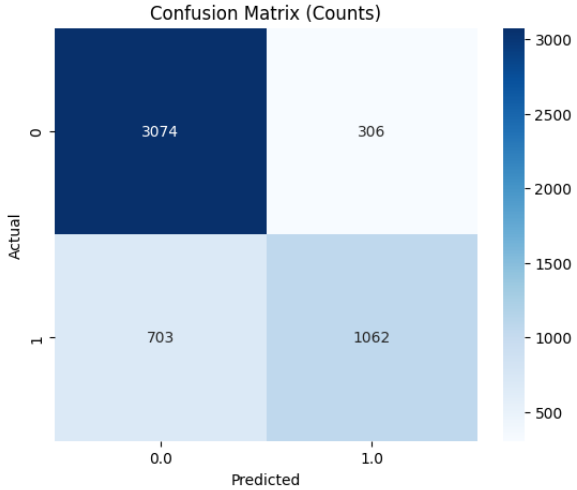


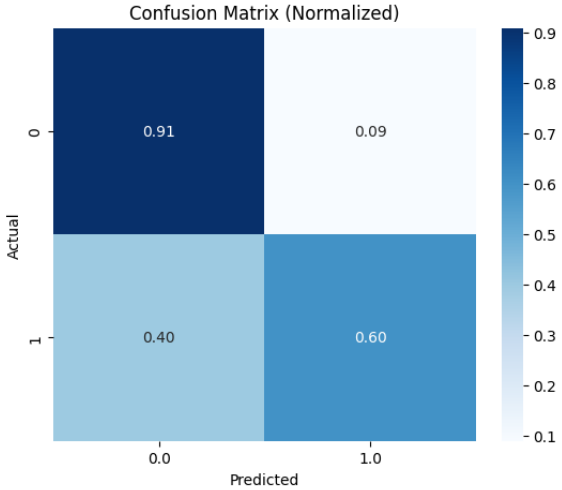


Random Forest Classifier:









12. Model Comparison

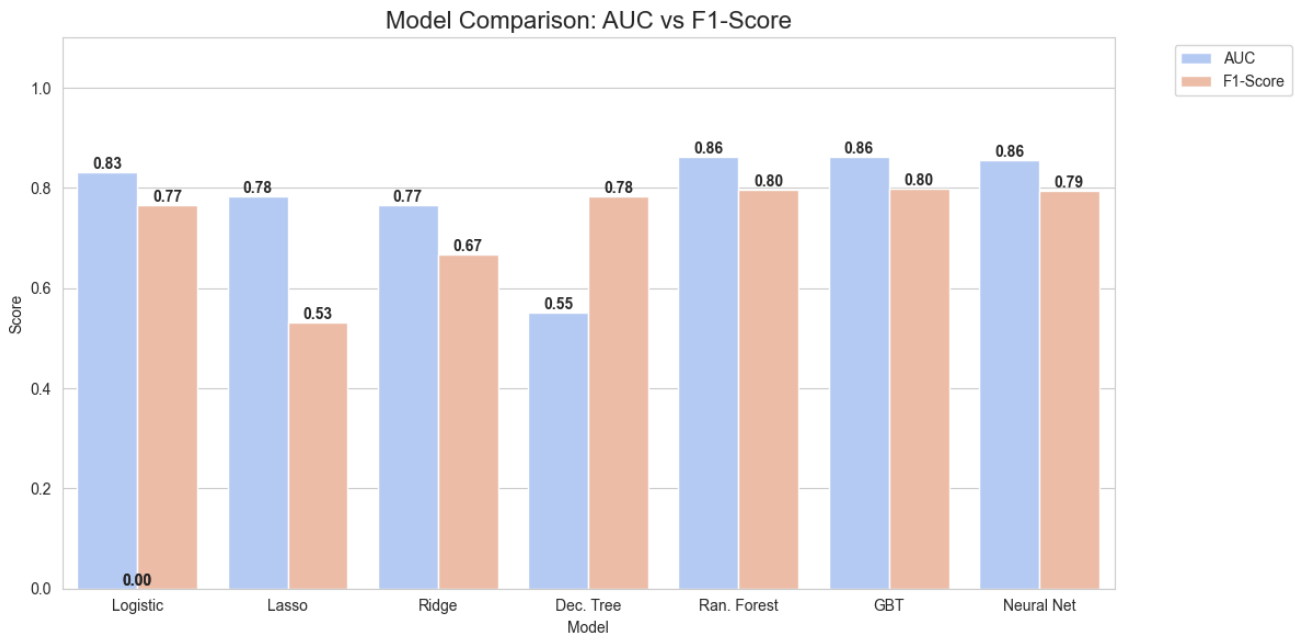
After training multiple classification models, we compared their performance to identify the most effective algorithm for predicting isBestSeller. The evaluation was based on the **test dataset** using key metrics such as **accuracy, precision, recall, F1-score**, and **ROC-AUC**, which provide insight into both overall performance and handling of imbalanced classes.

The models compared include:

- Logistic Regression
- Decision Tree
- Random Forest
- Gradient Boosting
- Neural Network

The comparison allows us to see which model best captures the patterns in the dataset while generalizing well to unseen data. From this analysis, we can select the **best-performing model** for deployment or further fine-tuning.

A visual representation (e.g., a bar chart or table) of the model performance metrics can help highlight the differences clearly.



13. Best Classifier Model and why?

From our model comparison, **Gradient Boosted Trees (GBT)** emerged as the **best-performing model**. It achieved the highest scores across multiple evaluation metrics, including **AUC, accuracy, F1-score, precision, and recall**. This indicates that GBT not only accurately predicts bestsellers but also effectively handles the imbalanced nature of the dataset, correctly identifying both bestseller (Yes) and non-bestseller (No) products.

The strong performance of GBT demonstrates its ability to capture complex patterns and interactions among features such as stars, reviews, listPrice, boughtInLastMonth, savings_pct, is_high_discount, and is_highly_rated. Therefore, it is the preferred model for deployment in predicting bestseller products.

14. Conclusion

In this project, we conducted a comprehensive analysis and modeling of an Amazon product dataset to predict bestsellers. The workflow included **data exploration, preprocessing, feature engineering, handling class imbalance, and visualization** to understand the dataset and extract meaningful insights.

We created additional features such as is_high_discount, is_highly_rated, and savings_pct to enhance predictive power, and selected the most relevant features through **correlation analysis**. After splitting the dataset into training and test sets and applying **VectorAssembler** and **StandardScaler** within a **PySpark pipeline**, we trained multiple classification models including Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, and Neural Network.

Based on evaluation metrics such as **accuracy, precision, recall, F1-score, and AUC**, the **Gradient Boosted Trees** model demonstrated the best performance, effectively handling the class imbalance and capturing complex relationships between features and the target variable.

This project highlights the importance of **data preprocessing, feature engineering, and model evaluation** in building robust predictive models. The workflow and insights gained here can be further extended to larger datasets or used to improve other e-commerce prediction tasks.